

SCX v4.0 规划 — 通用模块化架构

生成时间：2026-06-26 来源：GPT 深度讨论（与 gpt 的对话 202606261332.md, 5814 行）核心命题：“框架还是需要更通用一些，避免一直加模块迭代麻烦，应该模块化，不同的行业可以调用相同的代码”

0. 愿景

SCX 是一个领域无关的状态条件专家性框架，通过声明式配置 + 编码器插件适配任意数据域。

这不是一个”工具箱”式的框架——而是通过统一决策结构 $x \rightarrow s(x) \rightarrow \{Q, N, D, R, V\} \rightarrow a \rightarrow o \rightarrow \text{update}$ 将不同行业的”数据价值判断”问题抽象为同一个数学问题。不同领域只需要更换状态编码器（encoder）和声明式配置（YAML），核心的评估、路由、动作逻辑完全复用。

SCX for Good Data, Good Models, Good Decisions.

1. 当前问题

1.1 每加一个领域写一个 adapter，代码重复

当前 SCX v0.3.0 已有 MLIP (ACE descriptor)、Health (MedMNIST/DINO)、CIFAR (ResNet) 三个方向的实验代码。但每个方向的 encoder 实现、数据

加载流程、评估 pipeline 都是**单独编写**的，接口不统一。新增一个领域需要从头实现：

- State encoder（有自己的接口约定）
- Expert wrapper（每个 expert 的调用方式不同）
- 数据价值评估逻辑（被硬编码在具体模块中）
- 实验 pipeline（每个方向一套脚本）

1.2 接口不统一

MLIP 的 encoder 用 ACE descriptor 输出向量，Vision 的 encoder 用 DINO/CLIP/ViT 输出 embedding，它们之间没有公共抽象基类。导致：

- `scx/state/` 中的 clustering/statistics 逻辑与具体 encoder 耦合
- `scx/valuation/` 中的价值函数假设了特定的状态表示格式
- `scx/expert/` 中的专家路由逻辑需要为每个领域单独适配

1.3 配置散落各处

- 阈值参数写在代码里（如 `error_high=0.05`）
- 专家注册信息分散在各个模块
- 数据集路径、模型参数、实验配置没有统一管理
- 无法通过一份配置文件完整描述一个 SCX 任务

1.4 商业上不可复制

当前的架构意味着每服务一个行业客户，都需要大量定制开发。无法快速生成“试点 → 审计报告 → 私有部署”的标准流程。

2. 目标架构

2.1 核心抽象：三层分离

SCX Framework Core
(state assessment, valuation, routing, action)

SCXStateEncoder (Abstract)

MLIP	Vision	Tabular	Robot
Encoder	Encoder	Encoder	Encoder

SCX Config (YAML / dict)
domains/ mlip.yaml, health.yaml, cifar.yaml

第一层：SCX Framework Core — 完全领域无关

- SCXStateEncoder 抽象基类：定义 featurize(), distance(), get_state(), state_statistics() 接口
- SCXExpert 抽象基类：定义 predict(), confidence(), cost() 接口
- SCXDataPoint 抽象基类：定义统一的 data point 接口
- 价值评估、冗余检测、噪声识别、专家路由——全部只依赖抽象接口
- OnlineSCXFramework 也使用抽象接口，不耦合具体 encoder

第二层：Encoders（插件式） — 领域特定

每个 encoder 继承 SCXStateEncoder，只需实现接口方法：

Encoder	输入	输出	方法
MLIPEncoder	原子构型	ACE/SOAP fingerprint	featurize()
VisionEncoder	图像	DINO/CLIP/ViT embedding	featurize()
TabularEncoder	表格数据	归一化特征向量	featurize()
TrajectoryEncoder	机器人轨迹	视觉-动作-接触 embedding	featurize()
SimulationEncoder	网格/几何	mesh embedding	featurize()
TextEncoder	文本	LLM embedding	featurize()

第三层：Config（声明式配置） — YAML 描述完整任务

每个领域一个 YAML 文件，包含：- 数据路径/格式 - encoder 类型和参数 - 专家定义（模型路径/调用方式）- SCX 阈值和策略参数 - 输出报告模板

2.2 统一决策结构

所有领域共享相同的核心逻辑：

$$\mathbf{x} \rightarrow \mathbf{s}(\mathbf{x}) \rightarrow \{\mathbf{Q}(\mathbf{x}|\mathbf{s}), \mathbf{N}(\mathbf{x}|\mathbf{s}), \mathbf{D}(\mathbf{x}|\mathbf{s}), \mathbf{R}_e(\mathbf{s}), \mathbf{V}(\mathbf{x}|\mathbf{s})\} \rightarrow \mathbf{a}^*(\mathbf{x}) \rightarrow \mathbf{o} \rightarrow \text{update}$$

也就是：- $\mathbf{x}$: 原始数据点（构型、图像、轨迹、仿真点）- $\mathbf{s}(\mathbf{x})$: 状态编码（通过 encoder 映射到状态空间）- $\mathbf{Q}$: 质量评分（数据完整性、信号质量）- $\mathbf{N}$: 噪声/异常风险（是否可能坏数据）- $\mathbf{D}$: 冗余度（该状态已有多少数据）- $\mathbf{R}_e(\mathbf{s})$: 专家 e 在状态 s 的可靠性 - $\mathbf{V}(\mathbf{x}|\mathbf{s})$: 数据点的条件价值 - $\mathbf{a}^*(\mathbf{x})$:

最优动作 (keep/compress/relabel/route/high-fidelity/monitor) - o:
动作后的真实结果反馈 - update*: SCX 在线校准和状态数据库更新

2.3 与 SCX-Monitor 的关系

通用模块化架构自然支持在线模式:

离线审计 (SCX-Audit) : 历史数据 → 状态编码 → 价值审计报告

在线监测 (SCX-Monitor): 实时数据流 → 状态编码 → 风险/可信度 → 动作建议

闭环校准 (SCX-Learn) : 动作 → 结果反馈 → 数据库更新 → 策略进化

三种模式共享相同的抽象接口, 只是输入源不同。

3. 分阶段路线

Phase A: 核心重构 (2-4 周) P0

目标: 定义抽象接口, 将现有代码解耦, 证明重构后的代码能通过 336 tests。

交付物: - [] SCXStateEncoder / SCXExpert / SCXDataPoint 抽象基类定义
- [] encoders/ 目录, 包含 mlip/ vision/ tabular/ 三个初始 encoder - []
现有 scx/state/ scx/expert/ scx/valuation/ scx/action/ 重构为只依赖抽象接口 - [] domains/ 目录, 包含 mlip.yaml health.yaml cifar.yaml
三个示例配置 - [] 所有 336 tests 通过, 无回归

具体任务:

1. 定义抽象基类 (3 天)

- SCXStateEncoder: featurize(), distance(), get_state(), state_statistics()
- SCXExpert: predict(), confidence(), cost(), metadata
- SCXDataPoint: features, label, meta, state
- 类型标注 + 文档字符串 + 抽象方法检查

2. 提取现有 encoder (3 天)

- 从 experiments/mlip_case/ 提取 MLIPEncoder (ACE descriptor)
- 从 scx-health/ 提取 VisionEncoder (DINO/CLIP/ViT)
- 从 experiments/cifar/ 提取 CIFAREncoder
- 每个 encoder 单独模块, 可独立 import

3. 重构核心模块 (5 天)

- scx/state/: state discovery 使用 SCXStateEncoder 接口
- scx/expert/: expert registry 使用 SCXExpert 接口
- scx/valuation/: 价值函数使用统一 DataPoint 和 State 类型
- scx/action/: 动作决策器使用抽象接口
- scx/core/online.py: OnlineSCXFramework 同样解耦

4. 创建声明式配置原型 (2 天)

- 定义 YAML schema
- 实现 ConfigLoader.from_yaml(path) → SCXConfig
- 三个示例配置: mlip.yaml, health.yaml, cifar.yaml

5. 验证 (2 天)

- 全部 336 tests 通过
 - 每个领域的 demo notebook 可运行
 - 代码覆盖率不下降
-

Phase B: 声明式配置 (2-3 周) P1

目标: 实现 YAML → SCXFramework 自动构建, CLI 接口。

交付物: - [] `scx run --config domains/mlip.yaml` CLI - [] 自动报告生成 (从配置 + 结果 → 审计报告模板) - [] 新增领域的步骤减少到: 写 YAML + 写 Encoder

具体任务:

1. **YAML 配置解析器完善** (3 天)
 - 支持嵌套专家定义、encoder 参数、阈值策略
 - 支持数据路径映射 (本地/SSHFS/远端)
 - 配置验证 + 错误提示
2. **Framework 自动构建** (3 天)
 - `SCXFramework.from_config(config)` 工厂方法
 - 自动注册 encoder、experts、策略模块
 - 自动构建实验 pipeline
3. **CLI 设计** (2 天)
 - `scx run` — 运行完整审计
 - `scx config-check` — 验证配置文件
 - `scx list-encoders` — 列出可用 encoder
 - `scx list-experts` — 列出已注册专家
4. **审计报告自动生成** (3 天)
 - 标准化报告模板 (状态覆盖图 + 冗余分析 + 噪声风险 + 专家可靠性矩阵 + 动作建议)
 - Markdown/PDF/HTML 输出
 - 可视化自动生成
5. **文档 + demo** (2 天)
 - 编写 encoder 开发指南
 - 编写 YAML 配置参考

- 端到端 demo notebook
-

Phase C: 新领域扩展 (4-8 周) P1

目标：用新的抽象接口快速适配 3 个新领域，验证框架的通用性。

交付物：- [] SCX-Robot: TrajectoryEncoder + robot.yaml - [] SCX-FEM: SimulationEncoder + fem.yaml - [] SCX-LLM: TextEncoder + llm.yaml (minimal)

具体任务：

1. SCX-Robot (2-3 周)

- 用 LeRobot 公开数据 / Open X-Embodiment 子集
- **TrajectoryEncoder:** 融合视觉 embedding + 关节状态 + 接触力 + 成功/失败信号
- 成功/失败状态分类、冗余轨迹压缩、失败状态挖掘
- 对比: full-data vs SCX-selected vs random 在 imitation policy 上的表现
- 不需真实机器人，只用公开数据集

2. SCX-FEM / PDE (2-3 周)

- 用 PDEBench 或自生成弹性/热传导数据
- **SimulationEncoder:** 网格/几何/响应 embedding
- 多保真调度: 粗网格 vs 细网格, surrogate 可靠性判断
- 对比: SCX 多保真调度 vs 全细网格 vs 全粗网格

3. SCX-LLM (minimal) (1-2 周)

- **TextEncoder:** LLM embedding + 任务类型编码
- 不是做通用 LLM judge，而是做”可验证任务的数据价值评估”
- 聚焦数学/代码/逻辑等有 verifier 的方向

- 极小验证：证明 SCX 可以区分有价值/冗余/噪声的 instruction 数据
-

Phase D: 插件系统 (长期) P2

目标：将 compress/online/influence 等模块重构为可插拔插件，建立社区贡献指南。

交付物：- [] 插件注册机制 - [] compress/online/influence 插件化 - [] 社区贡献 guide + CLA - [] SCX-Bench 标准化

具体任务：

1. 插件注册机制 (1 周)

- `scx.plugins.register()` 装饰器
- 插件发现：自动扫描 `scx_contrib/` 目录
- 插件沙箱：版本检查、依赖声明

2. 核心模块插件化 (2 周)

- `SCXCompress` 从硬编码变为插件
- `OnlineSCXFramework` 从硬编码变为插件
- `StateConditionedInfluence` 从硬编码变为插件
- 保留默认实现，允许第三方替换

3. 社区基础设施 (1 周)

- CONTRIBUTING.md
- CLA 模板
- Issue/PR 模板
- 开发者文档

4. SCX-Bench 标准化 (2 周)

- `benchmark.run(method, dataset) → results` API

- 5-10 数据集集成
- Baseline 集成: Random, Uncertainty, Diversity, Shapley, LESS, Coreset, SCX
- 自动 Leaderboard

4. 商业模式映射

从 GPT 讨论中提取不同行业的收费模式，统一为” 数据价值审计 + 私有部署” 三层产品结构。

4.1 行业产品矩阵

行业	产品	定价锚点	Pilot 报价
MLIP/DFT	SCX-Potential Compiler	节省核时（DFT 计算成本）	2-10 万
医学图像	SCX-Health Audit	节省标注成本	10-50 万
机器人	SCX-Robot Data Audit	节省遥操作/真机时间	10-30 万
工程仿真	SCX-FEM Scheduler	节省仿真时间（高保真计算）	5-15 万
半导体	SCX-Semi OPC Router	节省 EDA license 费用	30-100 万 (toy)
工业质检	SCX-Vision Audit	减少人工复检量	10-50 万

行业	产品	定价锚点	Pilot 报价
3D 打印	SCX-AM Audit	减少打样次数	10-30 万

4.2 三层收费结构

- 第一层：SCX Audit 审计报告

客户给数据 → 跑 SCX → 交报告 + selected dataset + 训练对比

收费：项目制（10-50 万/次）
- 第二层：SCX Private Deployment 私有部署

客户数据敏感 → 本地部署 SCX-Core + domain encoder + dashboard

收费：年费（50-300 万/年）
- 第三层：SCX Platform 平台授权

多项目使用 → API/CLI/dashboard + 企业数据不出域

收费：License（100-1000 万/年）

4.3 中立定位

SCX 的核心商业价值在于**中立性**——它是跨行业的数据价值审计标准层，不应被单一企业绑定。通过三层权益设计保护客户隐私和 SCX 核心：

层级	归属	是否进入 SCX 总库
原始数据	客户	不进入
脱敏状态统计	合同约定	可选择进入
通用算法改进	SCX	保留

5. 与论文谱系的关系

SCX v4 通用模块化架构将直接受益 5 篇论文谱系：

5.1 EGP Paper 1: ACE gauge merging

EGP Paper 1 的 ACE 专家合并经验直接启发了通用架构中的 SCXExpert 设计——专家不是黑盒，而是有 gauge、有 domain certificate、有 reliability map 的可组合模块。通用架构中的 Encoder 设计也受益于 ACE descriptor 的实际经验。

5.2 SCX-Theory

通用架构中的统一决策结构直接反映了 SCX-Theory 的数学命题：- 状态条件专家性（State-conditioned expertise）- 数据价值状态条件性（Value is state-conditioned）- 压缩保真定理（Compression fidelity）- 专家治理协议（Expert governance protocol）

5.3 SCX-MLIP

重构后的 MLIPEncoder 和 SCXExpert 接口将使 SCX-MLIP 实验更容易复现和扩展——不同势函数（ACE/NEP/MACE/DeepMD）统一通过 SCXExpert 接口注册，状态条件路由和蒸馏直接使用通用框架。

5.4 SCX-Sim / FEM

通用架构中的 `SimulationEncoder` 将 FEM 多保真调度变为” 换一个 encoder + 写一份 YAML” 的工作量。SCX-Sim 论文可以专注于” 多保真调度” 的科学贡献，而不是工程实现。

5.5 SCX-Health

`VisionEncoder` 统一了医学图像和自然图像的 encoder 接口。SCX-Health 实验可以在通用架构上直接运行，且可以自然扩展到工业视觉等其他图像领域。

6. 架构演进路径

v0.3.0（当前）	v0.4.0（Phase A）	v0.5.0（Phase B）	v0.6.0（Phase C）
紧耦合	抽象接口 + Encoder	YAML 驱动	插件系统
各领域独立pipeline	核心模块解耦	CLI 接口	社区贡献
配置散落代码	3 领域示例	自动报告	Benchmark
	336 tests 通过	encoder 开发指南	插件注册

7. 风险与缓解

风险	严重程度	缓解措施
过度抽象导致灵活性下降	中	保留 CustomEncoder 逃生口，允许直接操作原始特征
重构破坏现有实验复现	高	每个重构步骤保持 backward compat + test 覆盖
MLIP 特殊需求无法被通用接口覆盖	中	MLIP 特有的 gauge alignment 作为可选插件
新领域 encoder 开发成本高于预期	中	先用最简单的 embedding（如 flatten + PCA）做最小可行版本
论文审稿人质疑”框架论文”没有足够实验	中	论文重心放在具体实验（MLIP/Health/Sim），不单独发”框架 paper”
商业落地慢于预期	低	通用架构反而降低定制成本，加速 pilot 交付

8. 一句话总结

SCX v4 将从一个”各领域各写一套 adapter 的框架”进化成”换 encoder + 写 YAML 就能适配任意数据域的通用数据价值审计平台”，核心代码完全复用，商业交付效率提升 5-10 倍。